

Лекция 4: Циклы

Введение

На предыдущих лекциях мы научились писать программы, которые выполняют действия последовательно (одна строка за другой) и могут принимать решения с помощью ветвления. Но что, если нужно выполнить одно и то же действие много раз? Например:

- Вывести числа от 1 до 1000
- Посчитать сумму всех элементов массива
- Запрашивать пароль у пользователя до тех пор, пока он не введёт правильный

Для таких задач существуют **циклы**. Циклы позволяют многократно выполнять блок кода, пока выполняется определённое условие.

Сегодня мы изучим три вида циклов в Java:

- **for** — когда известно количество повторений
- **while** — когда количество повторений неизвестно, но есть условие
- **do-while** — когда нужно выполнить тело цикла хотя бы один раз

А также научимся управлять циклами с помощью `break` и `continue`.

1) Цикл `for`

Цикл `for` используется, когда **заранее известно**, сколько раз нужно повторить блок кода. Например: вывести числа от 1 до 10, заполнить массив из 100 элементов, вычислить факториал числа.

Синтаксис цикла `for`

```
for (инициализация; условие; изменение) {  
    // тело цикла — код, который будет повторяться  
}
```

Три части цикла `for`:

Часть	Назначение	Пример
Инициализация	Выполняется один раз в начале цикла. Обычно создаётся счётчик	<code>int i = 0</code>
Условие	Проверяется перед каждой итерацией.	<code>i < 10</code>

Часть	Назначение	Пример
	Если true — цикл продолжается, если false — завершается	
Изменение	Выполняется после каждой итерации. Обычно изменяет счётчик	i++

Таблица 1. Части цикла FOR

Простейший пример

```
public class SimpleFor {
    public static void main(String[] args) {
        // Вывод чисел от 1 до 5
        for (int i = 1; i <= 5; i++) {
            System.out.println("i = " + i);
        }

        System.out.println("Цикл закончен!");
    }
}
```

Вывод:

text

Копировать

Скачать

i = 1

i = 2

i = 3

i = 4

i = 5

Цикл закончен!

Как работает этот цикл:

1. `int i = 1` — инициализация (один раз)
2. `i <= 5` — проверка условия (`1 <= 5 → true`)
3. Выполняется тело цикла: `System.out.println("i = " + i)`
4. `i++` — увеличение счётчика (`i` становится 2)
5. Снова проверка условия (`2 <= 5 → true`) — повторяем
6. Когда `i` станет 6, условие `6 <= 5 → false` — выход из цикла

Различные варианты использования for

```

public class ForVariations {
    public static void main(String[] args) {
        // 1. Счёт от 0 до 9 (стандарт для массивов)
        for (int i = 0; i < 10; i++) {
            System.out.print(i + " ");
        }
        System.out.println(); // 0 1 2 3 4 5 6 7 8 9

        // 2. Счёт в обратном порядке
        for (int i = 10; i > 0; i--) {
            System.out.print(i + " ");
        }
        System.out.println(); // 10 9 8 7 6 5 4 3 2 1

        // 3. Шаг больше 1
        for (int i = 0; i <= 20; i += 5) {
            System.out.print(i + " ");
        }
        System.out.println(); // 0 5 10 15 20

        // 4. Шаг 2 (только чётные числа)
        for (int i = 2; i <= 10; i += 2) {
            System.out.print(i + " ");
        }
        System.out.println(); // 2 4 6 8 10

        // 5. Несколько переменных в инициализации
        for (int i = 0, j = 10; i < j; i++, j--) {
            System.out.println("i = " + i + ", j = " + j);
        }

        // 6. Пустые секции (но точки с запятой обязательны!)
        int k = 0;
        for (; k < 5; k++) {
            System.out.print(k + " ");
        }
        System.out.println(); // 0 1 2 3 4

        // 7. Бесконечный цикл (выход через break)
        // for (;;) {
        //     System.out.println("Бесконечный цикл");
        // }
    }
}

```

Область видимости переменной счётчика

Переменная, объявленная в инициализации цикла for, существует **только внутри цикла**:

```
public class ForScope {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++) {
            System.out.println("Внутри цикла: i = " + i);
        }
        // System.out.println(i); // ОШИБКА! Переменная i не видна здесь

        // Правильно: объявить переменную до цикла, если нужна после
        int j;
        for (j = 0; j < 5; j++) {
            System.out.println("j = " + j);
        }
        System.out.println("После цикла: j = " + j); // j = 5
    }
}
```

Типичные задачи на цикл for

```
public class ForProblems {
    public static void main(String[] args) {
        // Задача 1: Сумма чисел от 1 до N
        int n = 10;
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += i; // sum = sum + i
        }
        System.out.println("Сумма от 1 до " + n + " = " + sum); // 55

        // Задача 2: Факториал числа (5! = 1*2*3*4*5 = 120)
        int number = 5;
        int factorial = 1;
        for (int i = 2; i <= number; i++) {
            factorial *= i;
        }
        System.out.println(number + "! = " + factorial); // 120

        // Задача 3: Таблица умножения для числа 7
        int num = 7;
        System.out.println("Таблица умножения для " + num + ":");
        for (int i = 1; i <= 10; i++) {
```

```

        System.out.println(num + " × " + i + " = " + (num * i));
    }

    // Задача 4: Степень числа (2^5 = 32)
    int base = 2;
    int exponent = 5;
    int result = 1;
    for (int i = 0; i < exponent; i++) {
        result *= base;
    }
    System.out.println(base + "^" + exponent + " = " + result); // 32
}
}

```

2) Цикл while

Цикл while используется, когда **количество повторений заранее неизвестно**, но есть условие, которое проверяется **перед** каждой итерацией. Если условие изначально ложно, тело цикла не выполнится ни разу.

Синтаксис цикла while

```

while (условие) {
    // тело цикла — выполняется, пока условие истинно
}

```

Простейшие примеры

```

public class SimpleWhile {
    public static void main(String[] args) {
        // Пример 1: Вывод чисел от 1 до 5
        int i = 1;
        while (i <= 5) {
            System.out.println("i = " + i);
            i++; // БЕЗ ЭТОЙ СТРОКИ ЦИКЛ БУДЕТ БЕСКОНЕЧНЫМ!
        }

        // Пример 2: Сумма чисел от 1 до 100
        int sum = 0;
        int counter = 1;
        while (counter <= 100) {
            sum += counter;
            counter++;
        }
    }
}

```

```

System.out.println("Сумма от 1 до 100 = " + sum); // 5050

// Пример 3: Удвоение числа до превышения 1000
int x = 1;
int steps = 0;
while (x < 1000) {
    x *= 2;
    steps++;
    System.out.println("Шаг " + steps + ": x = " + x);
}
System.out.println("Потребовалось " + steps + " шагов");
}
}

```

Важное предупреждение: бесконечный цикл

Если забыть изменить переменную в условии, цикл станет бесконечным:

```

// БЕСКОНЕЧНЫЙ ЦИКЛ (НЕ ЗАПУСКАЙТЕ!)
int i = 0;
while (i < 10) {
    System.out.println("Привет!");
    // НЕТ i++ — значение i всегда 0, условие всегда true
}

```

Чтобы остановить бесконечный цикл в программе, используйте Ctrl + C в консоли или кнопку остановки в IDE.

Практические примеры с while

```

import java.util.Scanner;

public class WhilePractical {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Пример 1: Проверка пароля (пока не введут правильный)
        String correctPassword = "java123";
        String userPassword = "";

        while (!userPassword.equals(correctPassword)) {
            System.out.print("Введите пароль: ");
            userPassword = scanner.nextLine();
            if (!userPassword.equals(correctPassword)) {
                System.out.println("Неверный пароль! Попробуйте снова.");
            }
        }
    }
}

```

```

    }
}
System.out.println("Добро пожаловать!");

// Пример 2: Сумма положительных чисел (0 — выход)
int sum = 0;
int number;

System.out.println("Введите числа для суммирования (0 для выхода):");
number = scanner.nextInt();
while (number != 0) {
    sum += number;
    System.out.print("Ещё число: ");
    number = scanner.nextInt();
}
System.out.println("Сумма введенных чисел: " + sum);

// Пример 3: Проверка, что число находится в диапазоне
int age;
System.out.print("Введите ваш возраст (0-120): ");
age = scanner.nextInt();

while (age < 0 || age > 120) {
    System.out.print("Некорректный возраст! Введите число от 0 до 120: ");
    age = scanner.nextInt();
}
System.out.println("Ваш возраст: " + age);

scanner.close();
}
}

```

Когда использовать while, а когда for

Ситуация	Рекомендуемый цикл
Известно точное количество повторений	for
Нужен счётчик с шагом	for
Нужна переменная только внутри цикла	for
Количество повторений неизвестно заранее	while

Ситуация	Рекомендуемый цикл
Цикл зависит от действий пользователя	while
Цикл продолжается до выполнения условия	while

Таблица 2. Ситуации для использования циклов FOR и WHILE.

java

Копировать

Скачать

// Хорошо для for: фиксированное количество

```
for (int i = 0; i < array.length; i++) { ... }
```

// Хорошо для while: неизвестное количество

```
while (userInput != -1) { ... }
```

3) Цикл do-while

Цикл do-while очень похож на while, но с одним важным отличием: **условие проверяется после** выполнения тела цикла. Это означает, что тело цикла всегда выполнится **хотя бы один раз**, даже если условие изначально ложно.

Синтаксис цикла do-while

```
do {
    // тело цикла — выполняется хотя бы один раз
} while (условие);
```

Важно: После while (условие) обязательно ставится точка с запятой ;!

Сравнение while и do-while

```
public class WhileVsDoWhile {
    public static void main(String[] args) {
        int x = 10;

        // Цикл while: проверка перед выполнением
        System.out.println("=== Цикл while ===");
        while (x < 5) {
            System.out.println("while: x = " + x);
            x++;
        }
        System.out.println("while: цикл не выполнился ни разу");
    }
}
```



```

// Цикл do-while: проверка после выполнения
x = 10;
System.out.println("\n=== Цикл do-while ===");
do {
    System.out.println("do-while: x = " + x);
    x++;
} while (x < 5);
System.out.println("do-while: цикл выполнен один раз");
}
}

```

Вывод:

=== Цикл while ===

while: цикл не выполнен ни разу

=== Цикл do-while ===

do-while: x = 10

do-while: цикл выполнен один раз

Где применяется do-while

do-while идеально подходит для ситуаций, когда действие нужно выполнить **как минимум один раз**:

- Запрос данных у пользователя с проверкой (сначала ввели, потом проверили)
- Меню программы (сначала показали меню, потом ждём выбор)
- Игровой цикл (сначала выполнили ход, потом проверяем конец игры)

```
import java.util.Scanner;
```

```

public class DoWhileExamples {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

```

// Пример 1: Меню программы (выполняется хотя бы один раз)

```

int choice;
do {
    System.out.println("\n=== ГЛАВНОЕ МЕНЮ ===");
    System.out.println("1. Начать игру");
    System.out.println("2. Загрузить игру");
    System.out.println("3. Настройки");
    System.out.println("0. Выход");
    System.out.print("Ваш выбор: ");
    choice = scanner.nextInt();

```

```

switch (choice) {
    case 1:
        System.out.println("Начинаем новую игру...");
        break;
    case 2:
        System.out.println("Загрузка сохранений...");
        break;
    case 3:
        System.out.println("Настройки...");
        break;
    case 0:
        System.out.println("До свидания!");
        break;
    default:
        System.out.println("Неверный выбор!");
}
} while (choice != 0);

```

// Пример 2: Ввод числа в диапазоне (сначала вводим, потом проверяем)

```

int number;
do {
    System.out.print("Введите число от 1 до 10: ");
    number = scanner.nextInt();
    if (number < 1 || number > 10) {
        System.out.println("Ошибка! Число должно быть от 1 до 10.");
    }
} while (number < 1 || number > 10);
System.out.println("Вы ввели: " + number);

```

// Пример 3: Генерация числа до получения нужного

```

int secretNumber = 7;
int guess;
do {
    System.out.print("Угадайте число (1-10): ");
    guess = scanner.nextInt();
    if (guess != secretNumber) {
        System.out.println("Не угадали! Попробуйте ещё.");
    }
} while (guess != secretNumber);
System.out.println("Поздравляю! Вы угадали!");

```

```

scanner.close();

```

```

}
}

```

4) Управление циклами: break и continue

Иногда нужно изменить нормальное выполнение цикла: досрочно выйти из него или пропустить текущую итерацию. Для этого используются операторы break и continue.

Оператор break

break **немедленно завершает** цикл (или switch). Программа продолжает выполнение с первого оператора после цикла.

```
public class BreakExample {
    public static void main(String[] args) {
        // Пример 1: Выход при нахождении нужного числа
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                System.out.println("Нашли число 5! Выходим из цикла.");
                break;
            }
            System.out.println("i = " + i);
        }
        System.out.println("Цикл завершён досрочно\n");

        // Пример 2: Поиск первого чётного числа
        int[] numbers = {3, 7, 9, 12, 15, 18};
        int firstEven = -1;
        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] % 2 == 0) {
                firstEven = numbers[i];
                System.out.println("Первое чётное число: " + firstEven);
                break;
            }
        }

        // Пример 3: Бесконечный цикл с выходом по условию
        int count = 0;
        while (true) { // бесконечный цикл
            System.out.println("Итерация " + count);
            count++;
            if (count >= 5) {
                System.out.println("Достигнут лимит. Выход.");
                break;
            }
        }
    }
}
```

```
}
```

Оператор continue

continue **пропускает текущую итерацию** и переходит к следующей. Оставшийся код в теле цикла для этой итерации не выполняется.

```
public class ContinueExample {
    public static void main(String[] args) {
        // Пример 1: Вывод только нечётных чисел
        System.out.println("Нечётные числа от 1 до 10:");
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) {
                continue; // пропускаем чётные
            }
            System.out.print(i + " ");
        }
        System.out.println("\n");

        // Пример 2: Сумма только положительных чисел
        int[] numbers = {5, -2, 10, -7, 3, -1, 8};
        int sum = 0;
        for (int num : numbers) {
            if (num < 0) {
                System.out.println("Пропускаем отрицательное: " + num);
                continue;
            }
            sum += num;
            System.out.println("Добавили " + num + ", сумма = " + sum);
        }
        System.out.println("Сумма положительных чисел: " + sum);

        // Пример 3: Пропуск определённых значений
        System.out.println("\nЧисла от 1 до 20, кроме кратных 3:");
        for (int i = 1; i <= 20; i++) {
            if (i % 3 == 0) {
                continue;
            }
            System.out.print(i + " ");
        }
    }
}
```

Сравнение break и continue

Оператор	Действие	Аналогия
break	Полностью выходит из цикла	"Я закончил"
continue	Пропускает текущую итерацию и переходит к следующей	"Пропустим этот раз"

Таблица 3. Операторы прерывания и пропуска

// Наглядное сравнение

```

public class BreakVsContinue {
    public static void main(String[] args) {
        System.out.println("=== break при i == 3 ===");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                break; // выход из цикла
            }
            System.out.print(i + " ");
        }
        System.out.println("\nЦикл закончен\n");

        System.out.println("=== continue при i == 3 ===");
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // пропуск i=3
            }
            System.out.print(i + " ");
        }
        System.out.println("\nЦикл закончен");
    }
}

```

Вывод:

```

=== break при i == 3 ===
1 2
Цикл закончен

```

```

=== continue при i == 3 ===
1 2 4 5
Цикл закончен

```

Вложенные циклы и break

В случае вложенных циклов break выходит только из **самого внутреннего** цикла:

```
public class NestedBreak {
    public static void main(String[] args) {
        // break выходит только из внутреннего цикла
        for (int i = 1; i <= 3; i++) {
            System.out.println("Внешний цикл: i = " + i);
            for (int j = 1; j <= 5; j++) {
                if (j == 3) {
                    System.out.println(" break при j = " + j);
                    break; // выход из внутреннего цикла
                }
                System.out.println(" j = " + j);
            }
            System.out.println("После внутреннего цикла\n");
        }
    }
}
```

Если нужно выйти из всех вложенных циклов сразу, используют **метки** (labels):

```
public class BreakWithLabel {
    public static void main(String[] args) {
        // Метка для внешнего цикла
        outerLoop:
        for (int i = 1; i <= 3; i++) {
            System.out.println("Внешний цикл: i = " + i);
            for (int j = 1; j <= 5; j++) {
                if (i == 2 && j == 3) {
                    System.out.println(" break outerLoop при i=" + i + ", j=" + j);
                    break outerLoop; // выход из обоих циклов
                }
                System.out.println(" j = " + j);
            }
        }
        System.out.println("Программа продолжается после вложенных циклов");
    }
}
```

5) Вложенные циклы

Циклы можно вкладывать друг в друга. Внутренний цикл выполняется полностью на каждой итерации внешнего цикла.

Простые вложенные циклы

java

```
public class NestedLoops {
    public static void main(String[] args) {
        // Пример 1: Прямоугольник из звёздочек
        System.out.println("Прямоугольник 5×3:");
        for (int row = 1; row <= 3; row++) {    // внешний цикл (строки)
            for (int col = 1; col <= 5; col++) { // внутренний цикл (столбцы)
                System.out.print("* ");
            }
            System.out.println(); // переход на новую строку
        }

        // Пример 2: Таблица умножения
        System.out.println("\nТаблица умножения (от 2 до 5):");
        for (int i = 2; i <= 5; i++) {
            for (int j = 2; j <= 5; j++) {
                System.out.printf("%2d×%2d=%3d ", i, j, i * j);
            }
            System.out.println();
        }
    }
}
```

Вывод:

Прямоугольник 5×3:

```
* * * * *
* * * * *
* * * * *
```

Таблица умножения (от 2 до 5):

2×2=4	2×3=6	2×4=8	2×5=10
3×2=6	3×3=9	3×4=12	3×5=15
4×2=8	4×3=12	4×4=16	4×5=20
5×2=10	5×3=15	5×4=20	5×5=25

Геометрические фигуры

```
public class ShapesWithLoops {
    public static void main(String[] args) {
        // Фигура 1: Прямоугольный треугольник
        System.out.println("Прямоугольный треугольник:");
        for (int i = 1; i <= 5; i++) {
```

```

        for (int j = 1; j <= i; j++) {
            System.out.print("* ");
        }
        System.out.println();
    }

    // Фигура 2: Перевернутый треугольник
    System.out.println("\nПеревернутый треугольник:");
    for (int i = 5; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            System.out.print("* ");
        }
        System.out.println();
    }

    // Фигура 3: Числовая пирамида
    System.out.println("\nЧисловая пирамида:");
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            System.out.print(j + " ");
        }
        System.out.println();
    }

    // Фигура 4: Треугольник с пробелами (равнобедренный)
    System.out.println("\nРавнобедренный треугольник:");
    int height = 5;
    for (int i = 1; i <= height; i++) {
        // Пробелы
        for (int j = 1; j <= height - i; j++) {
            System.out.print(" ");
        }
        // Звездочки
        for (int j = 1; j <= 2 * i - 1; j++) {
            System.out.print("* ");
        }
        System.out.println();
    }
}

```

Вложенные циклы с break и continue

```

public class NestedWithControl {
    public static void main(String[] args) {

```



```

// Поиск пары чисел, дающих сумму 10
int[][] matrix = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

int targetSum = 10;
boolean found = false;

outerLoop:
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        for (int k = j + 1; k < matrix[i].length; k++) {
            if (matrix[i][j] + matrix[i][k] == targetSum) {
                System.out.println("Найдена пара: " + matrix[i][j] + " + " + " + matrix[
i][k] + " = " + targetSum);
                found = true;
                break outerLoop; // выходим из всех циклов
            }
        }
    }
}

if (!found) {
    System.out.println("Пара не найдена");
}
}

```

6) Типичные ошибки при работе с циклами

Ошибка 1: Бесконечный цикл

```

// ОШИБКА: переменная не изменяется
int i = 0;
while (i < 10) {
    System.out.println("Привет!");
    // НЕТ i++ → бесконечный цикл
}

```

// ПРАВИЛЬНО

```

int i = 0;
while (i < 10) {

```

```
System.out.println("Привет!");  
i++;  
}
```

Ошибка 2: Точка с запятой после цикла

```
// ОШИБКА: точка с запятой завершает цикл  
for (int i = 0; i < 5; i++); {  
    System.out.println("Выполнится один раз");  
}  
// Тело цикла — пусто, блок в {} не относится к циклу  
  
// ПРАВИЛЬНО  
for (int i = 0; i < 5; i++) {  
    System.out.println("Выполнится 5 раз");  
}
```

Ошибка 3: Неправильное условие выхода

```
// ОШИБКА: условие никогда не станет false  
for (int i = 0; i > 0; i++) { // i > 0 изначально false  
    System.out.println("Не выполнится");  
}  
  
// ОШИБКА: условие всегда true  
for (int i = 0; i >= 0; i++) { // i всегда >= 0  
    System.out.println("Бесконечный цикл");  
}
```

Ошибка 4: Забытый break в нужном месте

```
// НЕЭФФЕКТИВНО: цикл продолжается после нахождения результата  
int[] numbers = {1, 2, 3, 4, 5};  
int found = -1;  
for (int i = 0; i < numbers.length; i++) {  
    if (numbers[i] == 3) {  
        found = numbers[i];  
        // НЕТ break — цикл продолжается зря  
    }  
}  
  
// ЭФФЕКТИВНО: выход при нахождении  
for (int i = 0; i < numbers.length; i++) {  
    if (numbers[i] == 3) {
```

```
        found = numbers[i];
        break; // нашли — выходим
    }
}
```

Ошибка 5: Переполнение типа счётчика

```
// ОПАСНО: переполнение int
for (byte i = 0; i < 1000; i++) { // byte может хранить только до 127
    System.out.println(i); // после 127 пойдёт переполнение
}
```

```
// ПРАВИЛЬНО: использовать подходящий тип
for (int i = 0; i < 1000; i++) {
    System.out.println(i);
}
```

Итоги четвёртой лекции

Сегодня мы изучили все три вида циклов в Java:

Цикл for:

- Используется, когда известно количество повторений
- Содержит инициализацию, условие и изменение в одной строке
- Переменная счётчика обычно объявляется внутри цикла

Цикл while:

- Проверяет условие ПЕРЕД выполнением тела
- Может не выполниться ни разу
- Подходит для неизвестного количества повторений

Цикл do-while:

- Проверяет условие ПОСЛЕ выполнения тела
- Выполняется хотя бы один раз
- Идеален для меню и ввода с проверкой

Управление циклами:

- break — немедленный выход из цикла
- continue — пропуск текущей итерации

Ключевые выводы:

1. Всегда следите, чтобы условие цикла когда-нибудь стало false
2. Выбирайте цикл в зависимости от задачи
3. Используйте break для досрочного выхода
4. Вложенные циклы позволяют работать с таблицами и матрицами
5. Избегайте бесконечных циклов — они "зависят" программу